

Habit Tracker (Simple)

Bulut Mimarilerinde Test Muhendisligi — Final Rapor

Ogrenci	Mert Baytas
Ders	MTH2526-B25 — Bulut Mimarilerinde Test Muhendisligi
Egitmen	Busra Ayaksiz
Konu	#3 Habit Tracker API
Teslim	4 Haziran 2026
Repo	github.com/baytasmert/habit-tracker-simple

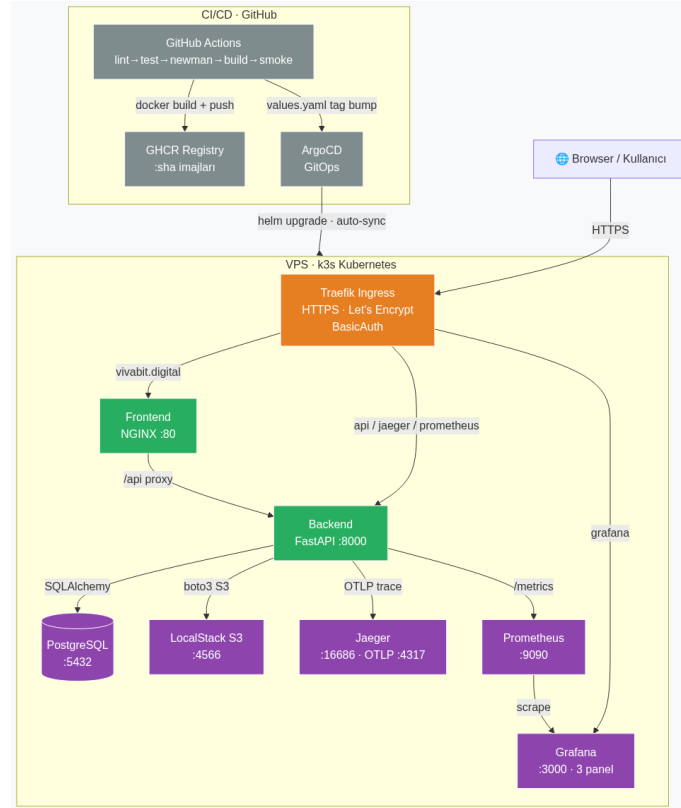
1. Giriş

Bu proje, Marmara Üniversitesi Bilgisayar Mühendisliği bölümünün "Bulut Mimarilerinde Test Mühendisliği" dersi kapsamında geliştirilmiş bireysel dönem projesidir. Konu olarak #3 **Habit Tracker API** seçilmiştir: günlük alışkanlık takibi, streak hesaplama ve REST API. Bu, aynı hesabın "habit-tracker-api" reposunun lightweight versiyonudur — sarnamenin minimum gereksinimlerini karşılayan, savunulması kolay basit bir mimari kurulmuştur.

Amac uygulamayı karmaşıklaştırmak değil, endüstri standardında **test ve dağıtım altyapısını** kurmaktır. Uygulama 14 endpoint (auth, habit CRUD, günlük track + mood, S3 foto) ile sade tutuldu; mimari ise tüm sarname katmanlarını (Docker, K8s/Helm, CI/CD, monitoring, perf, E2E) içerir.

2. Mimari

Sistem iki bağımsız servis olarak tasarlanmıştır: **frontend** (NGINX, sadece static dosya) ve **backend** (FastAPI, sadece JSON REST). Bu ayırım, gerçek üretim mimarilerinin ana özelliğidir; ayrıca her bir servisin bağımsız ölçeklenebilmesini sağlar.



Bileşen	Teknoloji	Port	Rol
Frontend	NGINX 1.25 + Vanilla JS	80	Static HTML/CSS/JS serve
Backend	FastAPI 0.110 + Python 3.11	8000	REST API (JSON)
DB	PostgreSQL 16	5432	User, Habit, HabitLog
S3 (LocalStack)	LocalStack 3	4566	Habit progress photo
Tracing	Jaeger 1.55	16686	OTLP span (bonus +5)
Metrics	Prometheus 2.51	9090	API metrik scrape
Dashboard	Grafana 10.4	3000	12 panel (API + host + kapasite)

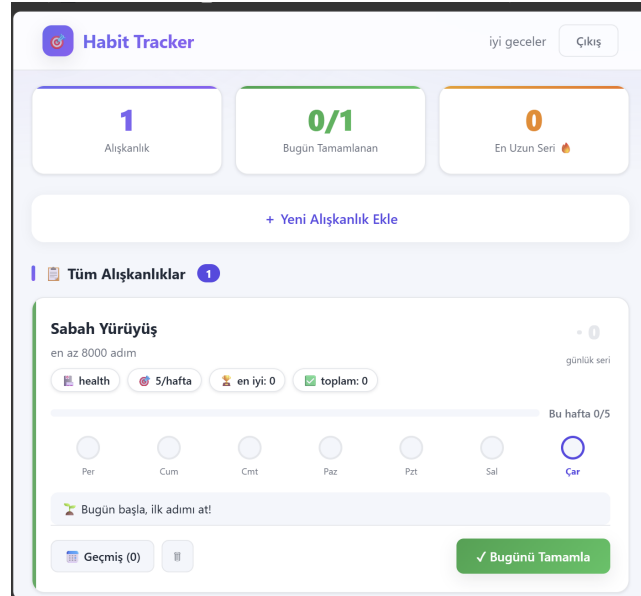
Uretim ortami: tek VPS üzerinde **k3s** cluster, **Helm** chart ile deploy, **Traefik** ingress (HTTPS / Let's Encrypt) ve **ArgoCD** ile GitOps otomatik senkronizasyon. Tum servisler vivabit.digital alt alanlari üzerinden yayınlanir.

2.1 Frontend ve Backend

Frontend (NGINX + vanilla JS): landing, login, register ve dashboard sayfalari. Dashboard ozet kartlari (toplam / bugun tamamlanan / en uzun seri), kategori renkli habit kartlari, haftalik ilerleme cubugu, 5 emoji mood secici ve fotograf yukleme sunar. Backend HTML uretmez; iletisim NGINX */api* reverse proxy üzerinden (tek origin, CORS yok).

Backend (FastAPI): yalniz JSON REST, 14 endpoint, JWT + bcrypt auth. Ana endpoint ler:

Method	Path	Aciklama
POST	/register · /login	Kayit + JWT token
GET / POST	/habits	Habit listele / olustur
POST	/habits/{id}/track	Gunu isaretle (UPSERT + mood)
GET	/habits/{id}/streak · /logs	Seri + toplam, gunluk gecmis
POST / GET	/habits/{id}/photo(s)	S3 foto yukle / listele / görüntüle
GET	/health · /metrics · /docs	Saglik, Prometheus, Swagger



Uygulama arayuzu — dashboard (vivabit.digital)

3. Test Stratejisi

Test piramidi yaklasimi: cok sayida birim test, orta sayida entegrasyon, az sayida E2E. Toplam testler ve hedef coverage:

Katman	Arac	Adet	Aciklama
Unit	pytest	24	auth + model + Factory Boy
Integration	pytest + TestClient	32	Endpoint + auth + log/mood + photo
Testcontainers	testcontainers-python	3	Gercek PostgreSQL container
E2E	Playwright	6	Tarayicida register/login/track/logout
API	Postman/Newman	7	CI da ko■ar
Perf	k6	2	Smoke (3VU/15s) + Load (20VU/60s)

3.1 Coverage Hedefi

Hedef >= %70. Olcum: %87 (src/auth, models, schemas, main coverage). pyproject.toml icinde --cov-fail-under=70 ile CI da zorunlu kilindi. Toplam birim+entegrasyon test sayisi: 56.

3.2 Factory Boy + Faker

tests/factories.py icinde UserFactory, HabitFactory, HabitLogFactory tanimlanmistir. Her test izole, gercekci ama unique veri kullanir.

4. Pipeline & Deploy

4.1 GitHub Actions

.github/workflows/ci.yml — tek workflow, 6 job zincirli:

Job	Amac	Onkos
lint	flake8 kod stili kontrolu	-
test	pytest + coverage 70+%	lint
newman	Postman koleksiyonu canli API ye (Postgres service)	lint
build	Docker (backend+frontend) -> GHCR (:sha, :latest)	test, newman
deploy-smoke	Kind cluster + helm template apply + curl + k6	build
cd-bump	values.yaml imaj tag bump -> commit (ArgoCD tetikler)	deploy-smoke

Fail-fast zincir: bir job kilirsa sonrakiler calismaz — bozuk kod build/deploy edilmez. build imajlari GHCR a (:sha) pushlar; deploy-smoke gercek bir Kind cluster da deploy edip dogrular (kalite kapi).

4.2 Deploy Stratejisi (GitOps)

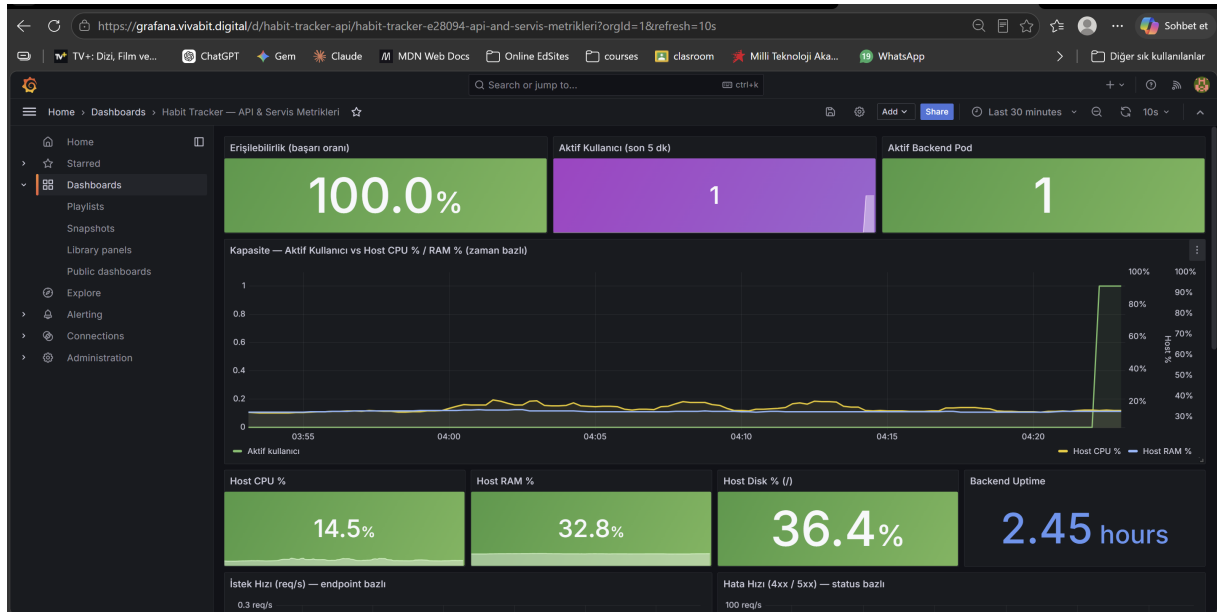
7 servisin tum K8s kaynaklari tek bir **Helm chart** ta toplandi; values.yaml sayesinde ayni chart lokal / CI / prod ta calisir (imaj tag, domain, replica oradan). Deploy **GitOps** tir: **ArgoCD** git i surekli izler, fark olunca cluster i otomatik sync eder. Rollback = onceki Git revizyonu veya :sha etiketli onceki imaj.

- **Yeni guncelleme akisi:** kod degisikligi → PR (CI tum testleri kosar, prod a dokunulmaz) → merge → imaj GHCR a push + cd-bump values.yaml a yeni :sha yazar → ArgoCD git i gorup k3s e rolling update ile uygular. Manuel kubectl yok; tek dogruluk kaynagi Git.

5. Monitoring ve Performans

5.1 Grafana Dashboard (12 panel)

Panel	Ne gosterir (kisaca)
Ozet kartlari	Erisilebilirlik %, aktif kullanıcı (5dk), saglikli pod
Kapasite	Aktif kullanıcı vs host CPU/RAM (zaman bazli korelasyon)
Host CPU/RAM/Disk + Uptime	VM kaynak kullanimi + backend ayakta suresi
Istek / Hata / Gecikme	Endpoint req/s, 4xx-5xx orani, p95/p99 latency
Gercek API Yuku	Endpoint bazli yuk dagilimi (probe/scrape haric)



Grafana — API & servis metrikleri (grafana.vivabit.digital)

5.2 k6 Sonuclari

Test	VU	Sure	p(95)	Hata	Sonuc
Smoke	3	15s	~5 ms	0%	PASS
Load	10->20	60s	~285 ms	0%	PASS

Yorum: p(95)=285 ms hedef olan 500 ms in cok altinda. En yavas endpoint ler /register ve /login — bcrypt password hashing guvenlik gereksinimi, optimize edilmez. Detayli rapor: perf/report.md.

6. Sonuc ve Ogrendiklerim

Olcut	Deger
Toplam Test	56 birim+entegrasyon + 3 TC + 6 E2E + 7 Newman
Test Coverage	%87 (CI gate: --cov-fail-under=70)
p(95) Latency	~285 ms (k6 load)
Build Suresi	~1 dk (backend + frontend imaj -> GHCR)
CI Pipeline	~5-6 dk (6 job: lint -> cd-bump)

Prod Deploy	ArgoCD otomatik sync ~1-3 dk (cd-bump sonrası)
Docker Image	Multi-stage backend + NGINX alpine frontend
K8s Deploy	Helm chart -> 7 servis (k3s prod + Kind CI)
Grafana Panel	12 panel (API + host + kapasite)
Bonus	Helm +5, OpenTelemetry +5, ArgoCD +5 (tavan +15)

6.1 Öğrendiklerim ve Zorluklar

- **Gözlemlenebilirlik tuzagi:** Grafana /health i 1-2K istek/s gösteriyordu, ama gerçek hız ~0.2/s idi. Sebep: Prometheus Service i scrape edince 2 replikanın sayıları tek seriye karışıp rate() sahte spike üretiyordu. Her pod u ayrı scrape (kubernetes_sd + RBAC) ile çözüldü. Metriğe korkmadan önce doğrulamak gerektiğini öğrendim.
- **Kapasite / CPU throttling:** k6 yukunde p95 13 s e fırladı ama host CPU sadece %37 idi -> darboğaz makine değil, pod un 0.5 CPU limiti (Kubernetes throttling) + her iterasyonda bcrypt. CPU limitini artırıp yük testini gerçekçi yaparak (VU basına tek login) çözdüm.
- **Metrik kardinalitesi:** Etiketler ham URL (/habits/1/track) her id yi ayrı seri yapıp kardinaliteyi patlatıyordu. Route şablonu (/habits/{habit_id}/track) ile tek seride topladım.
- **Kalıcılık:** Postgres ve Prometheus başlangıçta volume suz idi -> restart ta veri/metrik gidiyordu. PVC ekleyerek kalıcı kildim (stateless pod ile kalıcı state ayırımı öğrendim).
- **Konfigurasyon yönetimi:** Servis URL leri, DB ve S3 ayarlarını kod içinde yönetmek (lokal vs CI vs prod) çok zordu. Tek kaynaktan okumaya geçtim: pydantic-settings + .env / Helm values -> aynı kod her ortamda, sadece değer değişir.
- **Lint dengesi:** Basit stil kuralları (E203/E302/F401...) CI yi sürekli takiyordu. backend/.flake8 ile pragmatik bir ignore listesi hazırladım — kodu kiran hatalar yakalanır, stil takintileri gecilir.